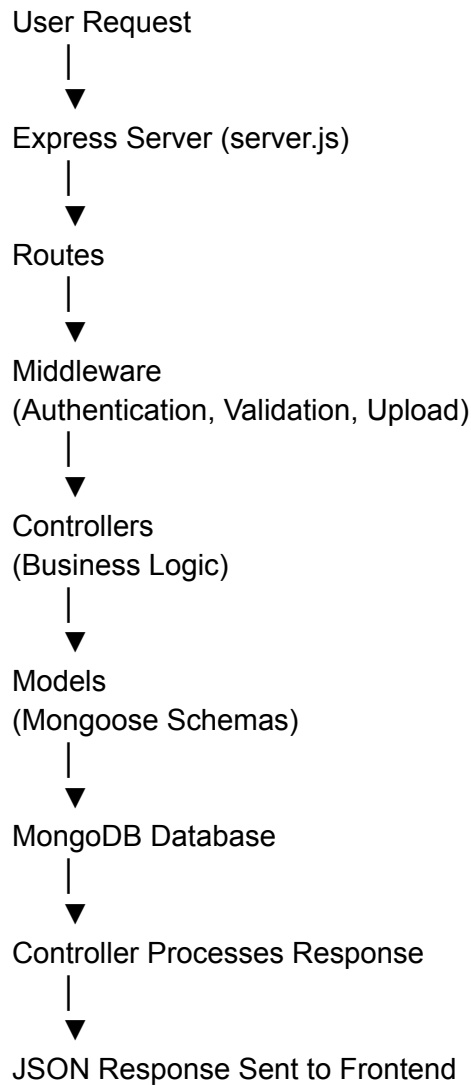


# Backend Request Flow



## Backend (Server) Development & Explanation

The backend of the **Online Complaint Registration System** is developed using **Node.js**, **Express.js**, and **MongoDB** following the **MVC (Model-View-Controller)** architecture. This architecture separates business logic, database operations, API routing, and middleware, making the project scalable, maintainable, and easy to manage.

---

# 1. Config Folder

## Purpose

The **config** folder contains all application configuration files. These files initialize external services such as the database and email server.

## Development

### db.js

- Connects the Express application with MongoDB.
- Uses Mongoose for database connectivity.
- Handles successful connection and database errors.

### Responsibilities

- Database connection
  - Error handling
  - Export database instance
- 

### mailConfig.js

This file configures the email service using **Nodemailer**.

### Used for

- Sending OTP during registration
  - Password reset emails
  - Complaint status notifications
  - Complaint resolution messages
- 

# 2. Controllers Folder

## Purpose

Controllers contain the complete business logic of the application.

They receive requests from routes, process data, interact with database models, and return responses.

---

## **authController.js**

### **Development**

Implements user authentication.

### **Functions**

- User Registration
  - Login
  - Logout
  - Forgot Password
  - Reset Password
  - JWT Token Generation
- 

## **userController.js**

### **Development**

Manages user-related operations.

### **Functions**

- View Profile
  - Edit Profile
  - Dashboard
  - Complaint History
- 

## **complaintController.js**

### **Development**

Core module of the system.

### **Functions**

- Register Complaint
  - Upload Complaint Image
  - View Complaint
  - Edit Complaint
  - Delete Complaint
  - Track Complaint Status
  - Assign Complaint
- 

## **adminController.js**

### **Development**

Handles administrator operations.

### **Functions**

- View All Users
  - View All Complaints
  - Assign Agents
  - Change Complaint Status
  - Generate Reports
  - Dashboard Analytics
- 

## **agentController.js**

### **Development**

Handles field agent operations.

### **Functions**

- View Assigned Complaints
  - Accept Complaint
  - Update Progress
  - Resolve Complaint
- 

## **feedbackController.js**

## Development

Collects user feedback.

### Functions

- Submit Rating
  - Submit Review
  - View Feedback
- 

## notificationController.js

### Development

Creates notifications.

### Functions

- Complaint Assigned
  - Complaint Accepted
  - Complaint Updated
  - Complaint Resolved
  - Read Notifications
- 

## 3. Middleware Folder

### Purpose

Middleware executes before controllers.

It validates requests and provides security.

---

## authMiddleware.js

### Development

Checks JWT authentication token.

## Responsibilities

- Verify Token
  - Authenticate User
  - Allow Authorized Access
- 

## roleMiddleware.js

### Development

Restricts pages according to user roles.

### Roles

- User
  - Agent
  - Admin
- 

## uploadMiddleware.js

### Development

Uses Multer package.

### Responsibilities

- Upload complaint images
  - Store uploaded files
  - Validate image format
- 

## validationMiddleware.js

### Development

Validates incoming request data.

### Checks

- Required Fields
  - Email Format
  - Password Strength
  - Complaint Information
- 

## **errorMiddleware.js**

### **Development**

Handles application errors globally.

Examples:

- Database Errors
  - Invalid Requests
  - Authentication Errors
  - Server Errors
- 

## **4. Models Folder**

### **Purpose**

Models define MongoDB collections using Mongoose Schema.

---

### **User.js**

Stores

- Name
  - Email
  - Phone
  - Password
  - Role
  - Address
-

## Complaint.js

Stores

- Complaint Title
  - Description
  - Department
  - Complaint Image
  - Complaint Status
  - Assigned Agent
  - Date
- 

## Agent.js

Stores

- Agent Name
  - Department
  - Contact Details
  - Assigned Complaints
- 

## Feedback.js

Stores

- Rating
  - Comments
  - Complaint ID
  - User ID
- 

## Notification.js

Stores

- Notification Message
- User ID
- Read Status



- Created Time
- 

## 5. Routes Folder

### Purpose

Routes define REST APIs.

Routes receive client requests and call controller functions.

---

### authRoutes.js

APIs

- POST Register
  - POST Login
  - POST Logout
  - POST Forgot Password
  - POST Reset Password
- 

### userRoutes.js

APIs

- GET Profile
  - PUT Update Profile
  - GET Dashboard
- 

### complaintRoutes.js

APIs

- POST Complaint
- GET Complaint

- PUT Update Complaint
  - DELETE Complaint
- 

## **adminRoutes.js**

APIs

- Manage Users
  - Manage Complaints
  - Dashboard
  - Reports
- 

## **agentRoutes.js**

APIs

- View Assigned Complaints
  - Update Complaint Status
  - Complete Complaint
- 

## **feedbackRoutes.js**

APIs

- Submit Feedback
  - View Feedback
- 

## **notificationRoutes.js**

APIs

- View Notifications
  - Mark Notification Read
-

## 6. Uploads Folder

### Purpose

Stores uploaded complaint files.

### Examples

- Complaint Images
  - Supporting Documents
  - Evidence Files
- 

## 7. Utils Folder

### Purpose

Contains reusable helper functions.

---

### **generateToken.js**

Creates JWT authentication token.

Used after successful login.

---

### **constants.js**

Stores

- User Roles
  - Complaint Status
  - Department Names
  - Response Messages
-

## 8. Environment File (.env)

### Purpose

Stores sensitive configuration information.

Contains

- MongoDB URL
  - JWT Secret
  - Email Username
  - Email Password
  - Server Port
- 

## 9. server.js

### Purpose

Main entry point of the backend application.

### Responsibilities

- Initialize Express Server
  - Connect MongoDB
  - Load Middleware
  - Register Routes
  - Start Server
- 

## 10. package.json

### Purpose

Maintains project dependencies.

Contains

- Project Name

- Version
  - Installed Packages
  - Scripts
  - Application Metadata
- 

## 11. package-lock.json

### Purpose

Locks dependency versions to ensure consistent installation across different environments.